Informe: servidor HTTP con Node.js

Actividad individual, módulo 1. Arquitectura Web, Universidad de Palermo, 2do cuatrimestre 2026.
Alumno: junam · Profesor: Ing. Diego Marafetti

Entorno y método de instalación

Sistema operativo	Windows 11 Home, versión 10.0.26200
Método de instalación	Instalación local del paquete LTS con el gestor de paquetes `winget` (`winget install OpenJS.NodeJS.LTS`), sin contenedor Docker
Versión de Node.js	v24.19.0 (LTS)
Versión de npm	11.17.0

Se optó por la instalación local en lugar de la dockerizada porque el equipo no tenía Docker instalado y la actividad no lo requiere: el runtime se va a usar durante toda la cursada, así que conviene tenerlo disponible directamente en el sistema. Se usó `winget` en lugar de bajar el instalador a mano porque resuelve la descarga y la incorporación al `PATH` en un solo comando.

Verificación de la instalación

```
> node --version
v24.19.0

> npm --version
11.17.0
```

Implementación

El servidor está en `modulo-1/server.js` y usa **exclusivamente el módulo nativo `http`**, sin Express ni ninguna otra dependencia. Se levanta con:

```
npm run modulo1
```

Expone dos rutas:

Verbo y ruta	Respuesta
`GET /`	`200` y el texto `Servidor funcionando correctamente`
`POST /archivo`	`200` y el texto `Bytes recibidos: <n>`
cualquier otra	`404` y el texto `Recurso no encontrado`

> ### Decisiones de implementación

El ruteo se hace a mano. Sin framework no hay tabla de rutas: lo único que identifica al pedido son `req.method` y `req.url`, y hay que compararlos dentro del callback de `http.createServer()`.

El conteo de bytes se acumula por fragmento. El cuerpo del pedido no llega entero ni de una sola vez: `req` es un *stream* legible y el contenido aparece fragmentado en *chunks* a medida que se recibe. Cada chunk es un `Buffer`, así que su `length` ya está en bytes y no en caracteres, que es la razón por la que el conteo funciona igual con contenido binario y con texto.

La respuesta se emite recién en el evento `end`. Contestar dentro del evento `data` devolvería un conteo parcial, porque en ese momento todavía pueden estar viajando fragmentos. Solo `end` garantiza que no va a llegar más contenido.

Se contempla el corte de la conexión. Si el envío se interrumpe a mitad de camino, el evento `error` del stream responde `400` en lugar de devolver un número incompleto como si fuera correcto.

Este ejercicio es el motivo por el que existe el header `Content-Length`: el servidor no sabe de antemano cuánto va a recibir, y tiene que contarlo o que se lo digan.

Evidencia de prueba

Se generó un archivo binario de 2048 bytes y se probaron las dos rutas con `curl`, más dos casos no contemplados para verificar el `404`.

```
> curl -s -w "[status: %{http_code}]" http://localhost:3000/
Servidor funcionando correctamente
[status: 200]

> curl -s -w "[status: %{http_code}]" -X POST --data-binary "@archivo-prueba.bin" http://localhost:3000/archivo
Bytes recibidos: 2048
[status: 200]

> curl -s -w "[status: %{http_code}]" http://localhost:3000/ruta-inexistente
Recurso no encontrado
[status: 404]

> curl -s -w "[status: %{http_code}]" -X POST --data "x" http://localhost:3000/
Recurso no encontrado
[status: 404]
```

El conteo coincide exactamente con el tamaño real del archivo, verificado con `(Get-Item archivo-prueba.bin).Length` = 2048.

> **Prueba con un archivo grande**

Con 2 kB el cuerpo entra en un solo fragmento, así que el conteo daría bien incluso con una implementación equivocada. Para verificar que la acumulación por chunks es realmente correcta se repitió la prueba con un archivo de 5.000.000 de bytes, que sí se parte en varios fragmentos:

```
> curl -s -w "[status: %{http_code}]" -X POST --data-binary "@archivo-grande.bin" http://localhost:3000/archivo
Bytes recibidos: 5000000
[status: 200]
```

Los archivos binarios de prueba están excluidos del repositorio por el `.gitignore` (`*.bin`), porque se generan al vuelo y no son código.

Dificultades encontradas

El `PATH` no se actualiza en las terminales ya abiertas. Después de instalar Node con `winget`, `node --version` seguía diciendo que el comando no existía. La instalación estaba bien: la variable `PATH` se modifica a nivel del sistema, pero las sesiones de terminal abiertas antes conservan la copia vieja. Se resuelve abriendo una terminal nueva.

En Windows PowerShell, `curl` no es `curl`. `curl` es un alias de `Invoke-WebRequest`, que tiene otra sintaxis y no acepta `--data-binary`. Para usar el `curl` real que viene con Windows hay que invocarlo como `curl.exe`.

Ninguna dificultad en la implementación del servidor en sí: la API de streams de `req` responde tal como está descripta en la documentación de Node.